Implementation of Simulated Annealing to Solve 8-Queens problem

```
import math
import random

def get_cost(state):
    attacks = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                attacks += 1
    return attacks

def get_neighbor(state):
    neighbor = list(state)
    col = random.randint(0, len(state) - 1)
    row = random.randint(0, len(state) - 1)
    neighbor[col] = row
    return neighbor

def simulated_annealing(n=8, t_init=100.0, alpha=0.99, t_min=0.001):
    current = [random.randint(0, n - 1) for _ in range(n)]
    current_cost = get_cost(current)
    t = t_init

    while t > t_min and current_cost > 0:
        neighbor = get_neighbor(current)
```

```
        neighbor_cost = get_cost(neighbor)
        delta_e = neighbor_cost - current_cost

        if delta_e < 0 or random.random() < math.exp(-delta_e / t):
            current = neighbor
            current_cost = neighbor_cost

        t *= alpha

    return current, current_cost

# Run
solution, cost = simulated_annealing()
print("Final State:", solution)
print("Conflicts (Cost):", cost)
for r in range(8):
    print(" ".join("Q" if solution[c] == r else "." for c in range(8)))
```

```
Final State: [4, 6, 3, 0, 2, 7, 5, 1]
Conflicts (Cost): 0
. . . Q . . . .
. . . . . . . Q
. . . . Q . . .
. . Q . . . . .
Q . . . . . . .
. . . . . . Q .
. Q . . . . . .
. . . . . Q . .
```

[ ]: